УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 22

МОДИФИКАЦИЯ ИМЕЮЩЕГОСЯ ПРОЕКТА. ЗАПУСК СОЗДАННОГО ПРИЛОЖЕНИЯ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Модификация имеющегося проекта. Запуск созданного приложения

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Отправка форм

Одной из форм отправки данных на сервер представляет отправка с помощью форм html, обычно в запросе типа POST. В Django в целом можно использовать два подхода для работы с формами. Во-первых, можно работать со стандартными формами html. Во-вторых, Django также предоставляет специальный функционал для работы с формами, который, возможно, в каких-то моментах упрощает работу с данными. В даннной главе рассмотрим оба подхода. А в данной статье посмотрим, как получать данные обычных форм html.

Обычно данные форм передаются на сервер в запросе типа POST. Для получения подобных данных в классе HttpRequest определено свойство POST. Например, пусть у нас есть следующий шаблон index.html:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>METANIT.COM</title>
</head>
<body>
    <h2>User form</h2>
    <form method="post" action="postuser/">
        {% csrf_token %}
        <p>Name:<br> <input name="name" /></p>
        <p>Age:<br> <input name="age" type="number" /></p>
        <input type="submit" value="Send" />
    </form>
```

```
</body>
</html>
```

Здесь определена форма условно для ввода данных пользователя, которая в запросе типа POST (атрибут method="post") отправляет данные по адресу "postuser/" (атрибут action="postuser/").

На форме определены два поля ввода. Первое поле предназначено для ввода имени пользователя. Второе поле - для ввода возроста пользователя.

Также внутри формы используется тег {% csrf_token %}. Он позволяет защитить приложение от CSRF-атак, добавляя в форму в виде скрытого поля csrf-токен. Кроме того, Django по умолчанию требует наличия данного токена в получаемых данных в запросе POST.

Для отправки формы и получения ее данных определим в файле views.py следующие функции:

```
from django.shortcuts import render
from django.http import HttpResponse
def index(request):
    return render(request, "index.html")
def postuser(request):
    # получаем из данных запроса POST отправленные через форму данные
    name = request.POST.get("name", "Undefined")
    age = request.POST.get("age", 1)
    return HttpResponse(f"<h2>Name: {name}  Age: {age}</h2>")
```

В представлении index возвращается шаблон, который содержит форму ввода.

В представлении postuser получаем через словарь request.POST отправленные из формы данные. В этом словаре по ключу можно получить значение элемента. При этом в качестве ключей выступает названия полей форм (значения атрибутов name элементов формы):

```
<input name="age" type="number" />
```

Так, в данном случае название поля (значение атрибута name) равно "age". Соответственно в request.POST по этому имени мы можем получить его значение:

```
age = request.POST.get("age", 1)
```

Если по каким-то причинам данные с ключом "age" в запросе отсутствуют, то возвращается значени по умолчанию - 1.

Далее в файле urls.py свяжем эти функции с маршрутами:

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path("postuser/", views.postuser),
]
```

И после получения данных формы они отправляются обратно клиенту (рисунок 1):

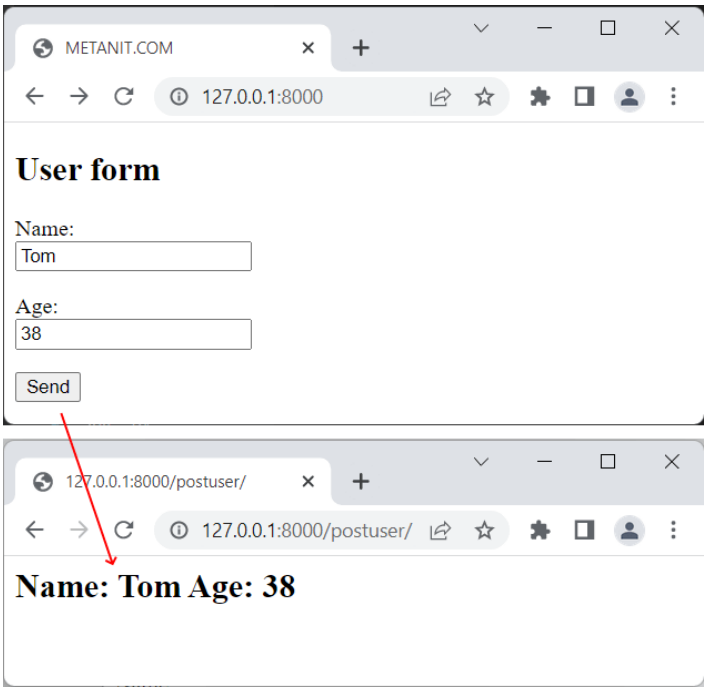


Рисунок 1 - Отправка форм в веб-приложении на Django и Python

4.2 Получение массивов

Усложним задачу и добавим в форму на странице index.html несколько полей, которые будут представлять массив:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>METANIT.COM</title>
</head>
<body>
    <h2>User form</h2>
    <form method="post" action="postuser/">
        {% csrf_token %}
        <p>Name:<br /> <input name="name" /></p>
        <p>Age:<br /> <input name="age" type="number" /></p>
        <p>
            Languages:<br />
            <input name="languages" /><br />
            <input name="languages" /><br />
            <input name="languages" /><br />
        </p>
        <input type="submit" value="Send" />
    </form>
</body>
</html>
```

Здесь практически та же форма, только добавлено три поля для ввода языка программирования. Причем каждое из этих полей имеет одно и то же имя - "languages". Благодаря этому при отправке формы в данных запроса будет сформирован список languages из данных, введенных в эти поля.

В файле views.py изменим функцию postuser для получения массива languages:

```
from django.shortcuts import render
from django.http import HttpResponse
def index(request):
    return render(request, "index.html")
def postuser(request):
    # получаем из строки запроса имя пользователя
    name = request.POST.get("name", "Undefined")
    age = request.POST.get("age", 1)
    langs = request.POST.getlist("languages", ["python"])
    return HttpResponse(f"""
            <div>Name: {name}  Age: {age}<div>
            <div>Languages: {langs}</div>
        """)
```

Ключевой компонент при получении списка данных из запроса представляет метод getlist(), который работает так же, как и get(), только возвращает список. Если в запросе не окажется данных с ключом languages, то возвращаем список ["python"]

Получив список, мы можем что-то сделать с его элементами, перебрать, обратиться к элементам по индексу и т.д. Но в данном случае просто передаем весь список в формируемый ответ (рисунок 2).

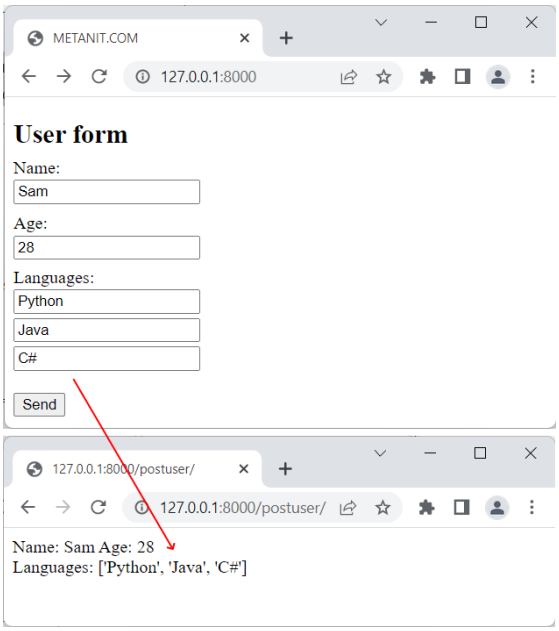


Рисунок 2 - Отправка массивов и списков в Django в запросе Post

Подобным образом можно передавать значения массива полей других типов, либо полей, которые представляют набор элементов, например, элемента select, который поддерживает множественный выбор (рисунок 3):

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>METANIT.COM</title>
</head>
<body>
    <h2>User form</h2>
    <form method="post" action="postuser/">
        {% csrf_token %}
        <p>Name: <br />
            <input name="name" />
        </p>
        <p>Age: <br />
            <input name="age" type="number" />
        </p>
        <p>
            Languages:<br />
            <select multiple name="languages">
                <option>Python</option>
                <option>JavaScript</option>
                <option>C++</option>
                <option>Java</option>
             </select>
        </p>
        <input type="submit" value="Send" />
    </form>
</body>
</html>
```

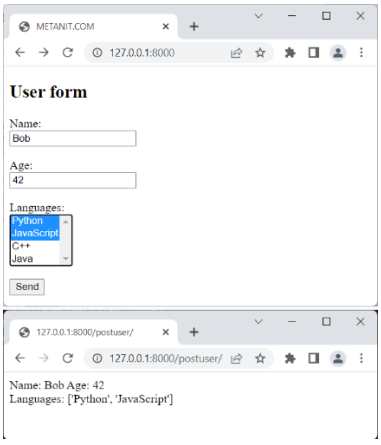


Рисунок 3 - Получение списков в Django в запросе Post

4.3 Создание форм и обработка запросов в Django

Django предоставляет ряд инструментов и библиотек, которые помогут вам создавать формы, принимающие ввод от посетителей сайта, а затем обрабатывающие вводимые данные и отвечающие на них.

В HTML, форма — это набор элементов внутри, **<form>...</form>,** которые позволяют посетителю делать такие вещи, как ввод текста, выбирать параметры, манипулировать объектами или элементами управления и т. Д., а затем отправлять эту информацию обратно на сервер.

Некоторые из этих элементов интерфейса формы - ввод текста или флажки - встроены в сам HTML. Другие намного сложнее; интерфейс, который открывает окно выбора даты или позволяет перемещать ползунок или управлять элементами управления, обычно использует JavaScript и CSS, а также **<input>**элементы формы HTML для достижения этих эффектов.

Помимо своих **<input>**элементов, форма должна определять две вещи:

– *где*: URL-адрес, по которому должны быть возвращены данные, соответствующие вводу пользователя;

– *как*: HTTP-метод, который должен возвращать данные

GET и POST

GET и **POST** являются единственными методами HTTP, которые можно использовать при работе с формами.

Форма входа в Django возвращается с использованием **POST** метода, в котором браузер объединяет данные формы, кодирует их для передачи, отправляет на сервер, а затем получает ответ.

GET, напротив, объединяет представленные данные в строку и использует ее для составления URL-адреса. URL-адрес содержит адрес, по которому должны быть отправлены данные, а также ключи и значения данных. Вы можете увидеть это в действии, если выполните поиск в документации Django, и в результате будет получен URL-адрес формы

GET и **POST** обычно используются для разных целей.

Любой запрос, который можно использовать для изменения состояния системы - например, запрос, который вносит изменения в базу данных - должен использовать **POST**. **GET** следует использовать только для запросов, не влияющих на состояние системы.

GET также не подходит для формы пароля, потому что пароль будет отображаться в URL-адресе и, следовательно, также в истории браузера и журналах сервера, все в виде обычного текста. Он также не подходит для больших объемов данных или для двоичных данных, таких как изображение. Веб-приложение, использующее **GET** запросы для форм администратора, представляет собой угрозу безопасности: злоумышленник может легко имитировать запрос формы для получения доступа к конфиденциальным частям системы. **POST**, в сочетании с другими средствами защиты, такими как защита Django от CSRF, обеспечивает больший контроль над доступом.

С другой стороны, **GET** он подходит для таких вещей, как форма веб-поиска, потому что URL-адреса, которые представляют **GET** запрос, можно легко добавить

в закладки, поделиться или повторно отправить.

Для начала создадим Django-проекта и полностью настроим отображение для страницы index.html (рисунок 4-5).



Рисунок 4 - Установка виртуального окружения



Рисунок 5 -Установка django

Далее, создадим новый проект django_forms_learning (рисунок 6):

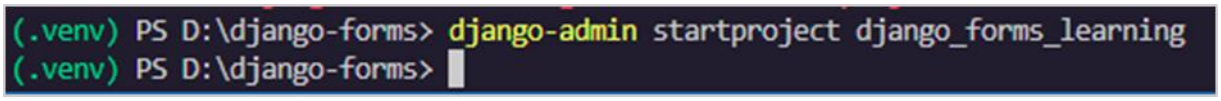


Рисунок 6 - Создание нового проекта django_forms_learning

Перейдем в неё (рисунок 7):

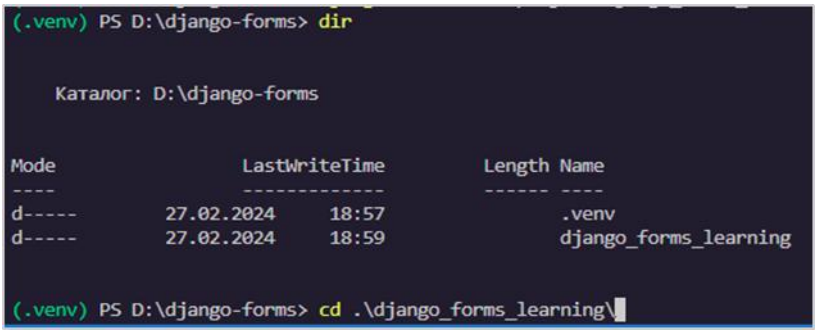


Рисунок 7 - Переход в директорию django_forms_learning

Создадим новое приложение. Назовем его index (рисунок 8):



Рисунок 8 - Создание нового django-app (django-приложения)

Зарегистрируем приложение в файле settings.py (в INSTALLED_APPS) (рисунок 9):

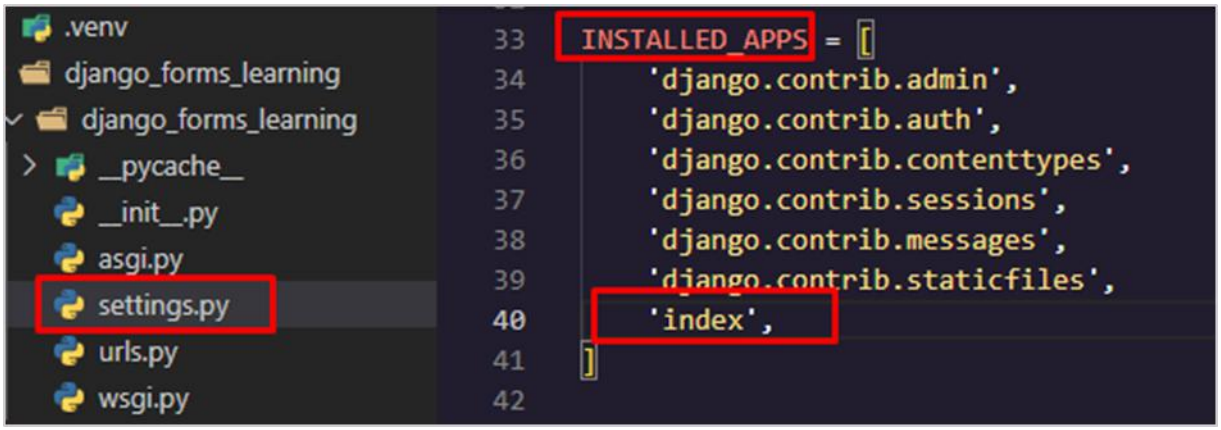


Рисунок 9 - Добавление в INSTALLED_APPS приложения index

Создадим константу, для хранения шаблонов (templates) (рисунок 10):



Рисунок 10 - Добавление директории для шаблонов (templates)

Мы использовали os.path, и для того, чтобы все работало, необходимо

импортировать модуль os. Для этого пропишем «import os» (рисунок 11):

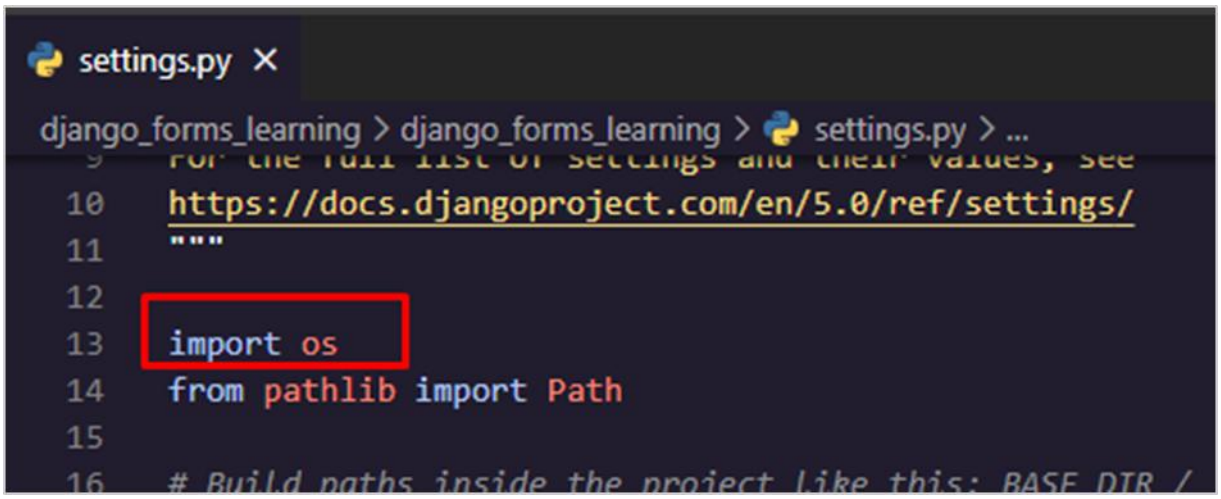


Рисунок 11 - Импортирование модуля os

Ваша иерархия проекта должна выглядеть следующим образом (рисунок 12):

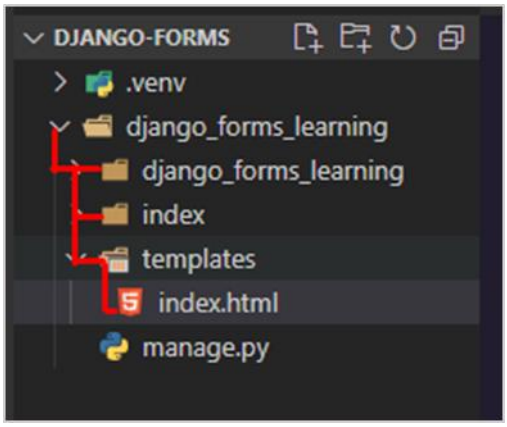


Рисунок 12 - Правильная иерархия проекта

Для красоты добавим стили Bootstrap через ссылку (CDN подключение) (рисунок 13):



Рисунок 13 - HTML-код с подключением Bootstrap через CDN

Далее, добавим вью-функцию для отображения страницы index.html (рисунок 14):

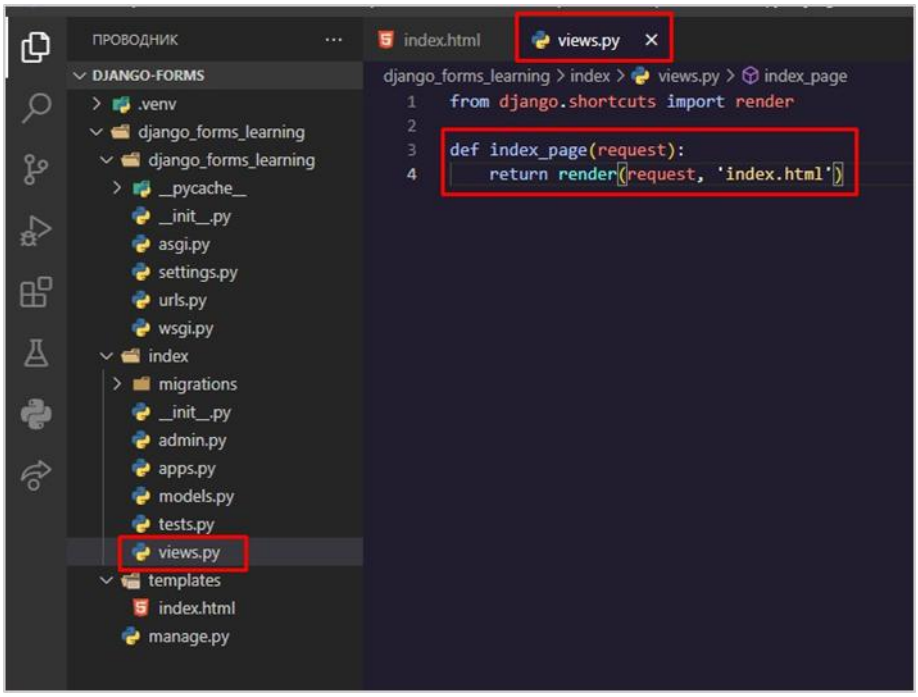


Рисунок 14 -Создание вью-функции для отображения главной страницы

Чтобы вью-функция работала, ей нужен задать маршрут. Зададим его на главную страницу, указав path('', index_page) (рисунок 15):

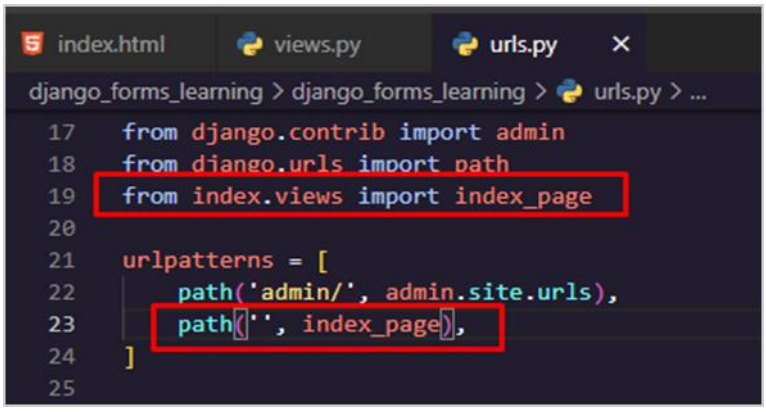


Рисунок 15 - Добавление маршрута в файле urls.py

В завершение, запустим локальный сервер (рисунок 16):

```
(.venv) PS D:\django-forms\django_forms_learning> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions
.
Run 'python manage.py migrate' to apply them.
February 27, 2024 - 19:07:39
Django version 5.0.2, using settings 'django_forms_learning.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 16 - Запуск локального сервера

Далее, после всех проделанных действий создадим форму. Для этого создадим в приложении index файл под названием forms.py и пропишем следующую логику (рисунок 17):

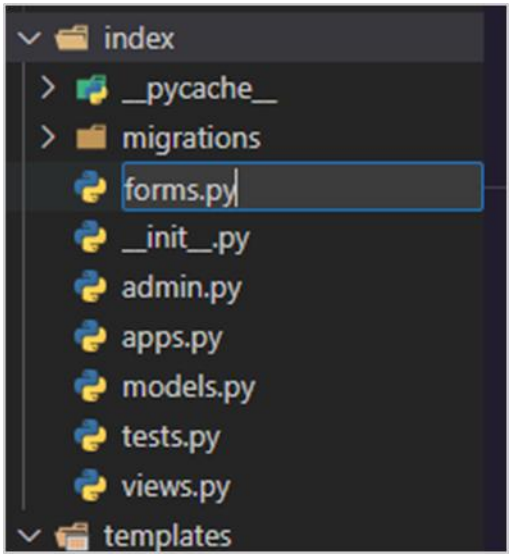


Рисунок 17 - Создание файла для форм (forms.py)

Изменим файл forms.py и добавим поля для ввода ФИО и возраста. Поля здесь вводятся по аналогии с моделями, только указывается не models, а forms (рисунок 18):

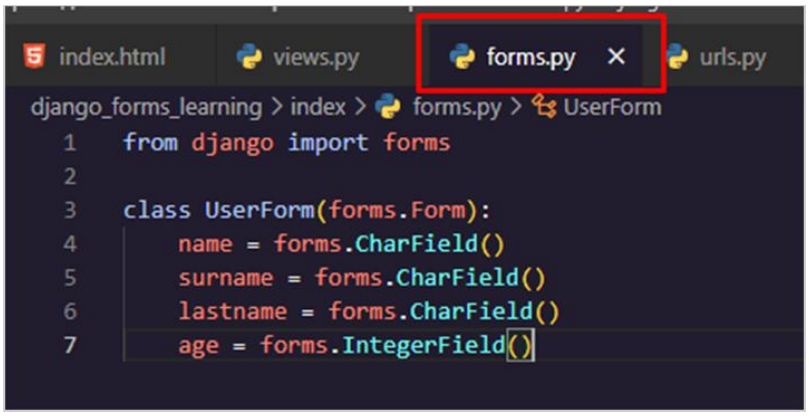


Рисунок 18 - Добавление полей для формы

Передадим форму в контекст, чтобы форма могла отображаться на странице. Для этого изменим вью-функцию, добавим импорт формы из приложения index, а также создадим переменную для хранения объекта формы UserForm (рисунок 19).

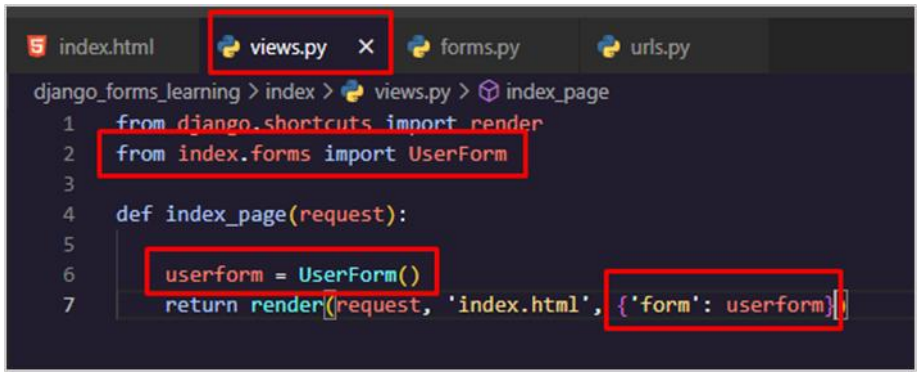


Рисунок 19 - Изменение вью-функции (добавление формы в контекст)

Вспоминая Django Template Language (DTL), отобразим форму на странице (рисунок 20):

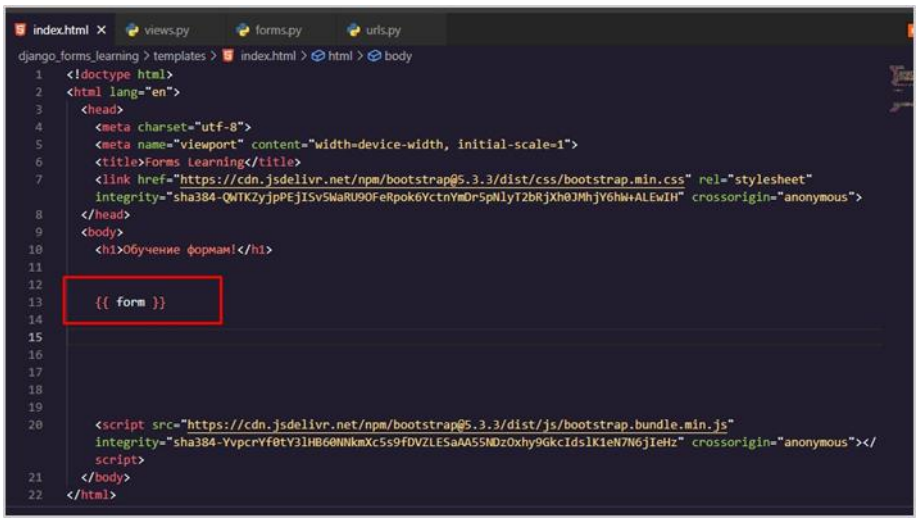


Рисунок 21 - Отображение формы на страницы, используя DTL

В конечном итоге, мы получим следующее (рисунок 22):

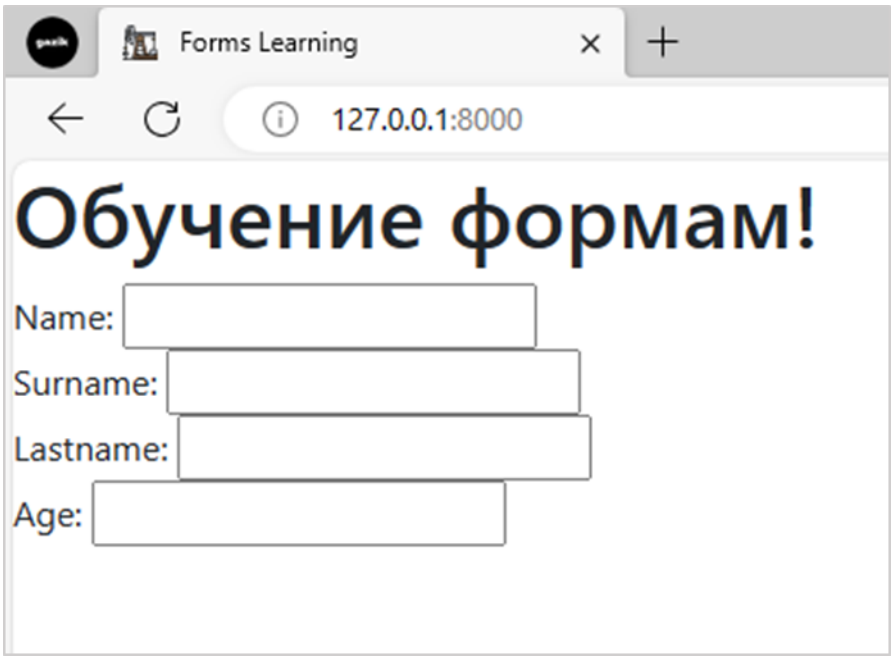


Рисунок 22- Форма, отображенная на странице

Чтобы отправлять данные на сервер с помощью, нужно установить атрибут method в форме на «post». По умолчанию используется метод GET. Также, необходимо добавить кнопку, либо поле input с атрибутом «submit» (рисунок 23).

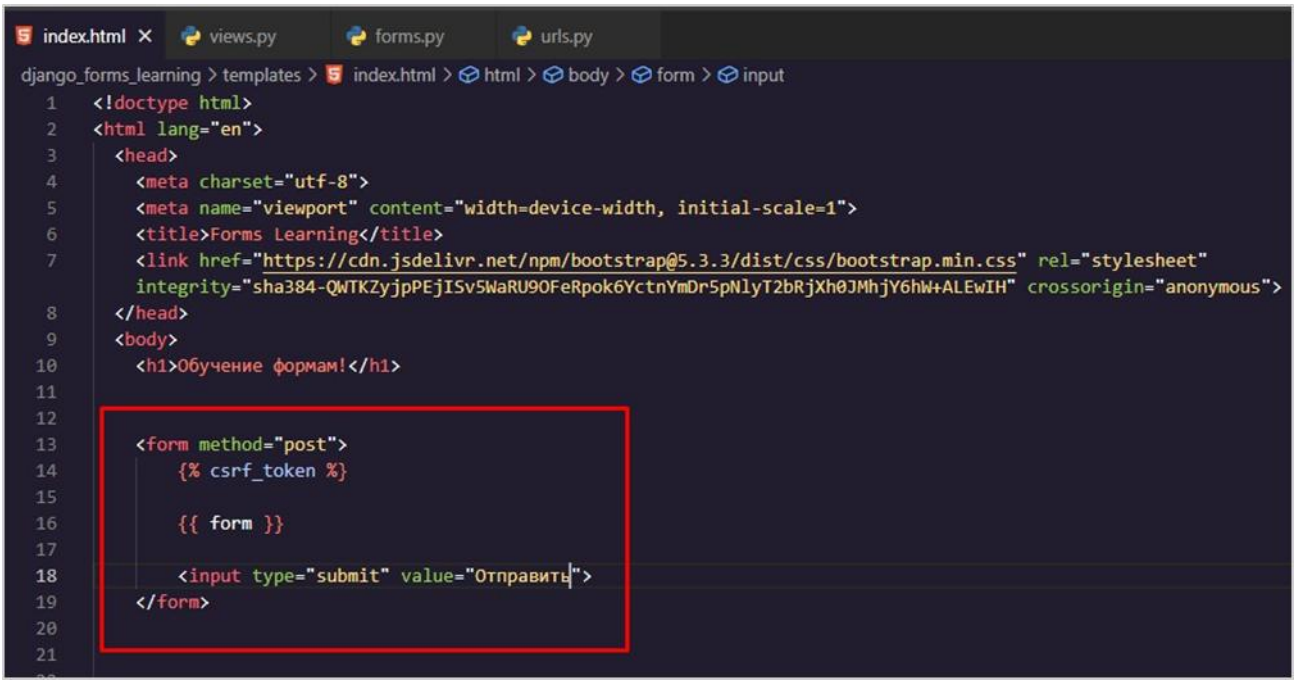


Рисунок 23 - Форма с post-запросом

Будем отправлять HTTP-ответ (HttpResponse) при отправке формы. Добавим проверку, что если метод нашего запроса POST, то у нас будет из полей формы с названиями name, surname, lastname и age браться значения и возвращаться пользователю в виде HttpResponse. Для этого изменим код следующим образом

(рисунок 24):

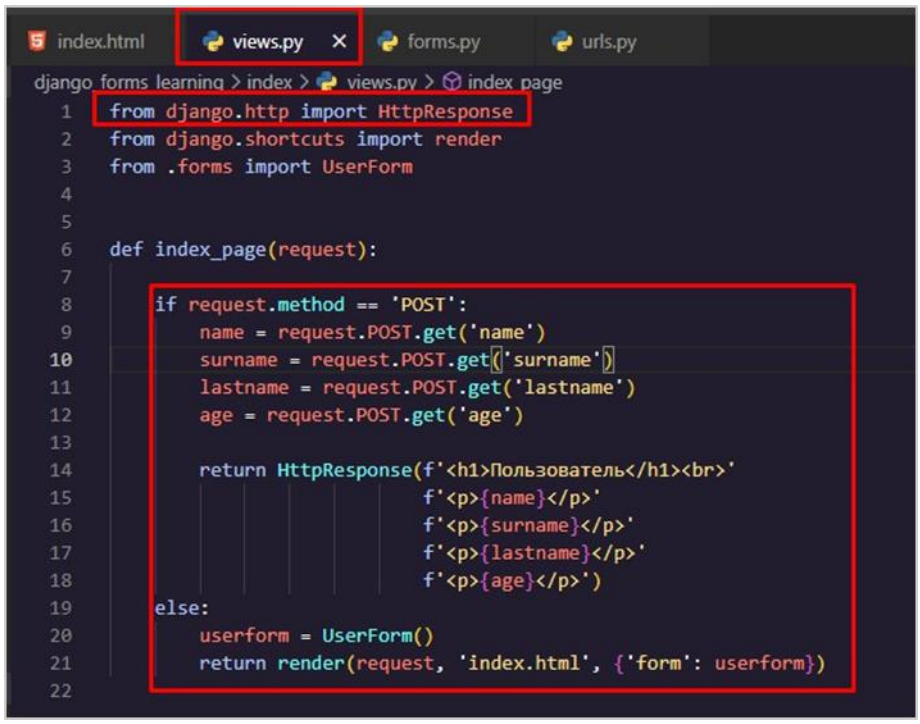


Рисунок 24 - HTTP-ответ от сервера пользователю (HTTP Response)

После проделанных действий, на странице отобразится форма с кнопкой отправить. Заполним её и нажмем «Отправить» (рисунок 25).

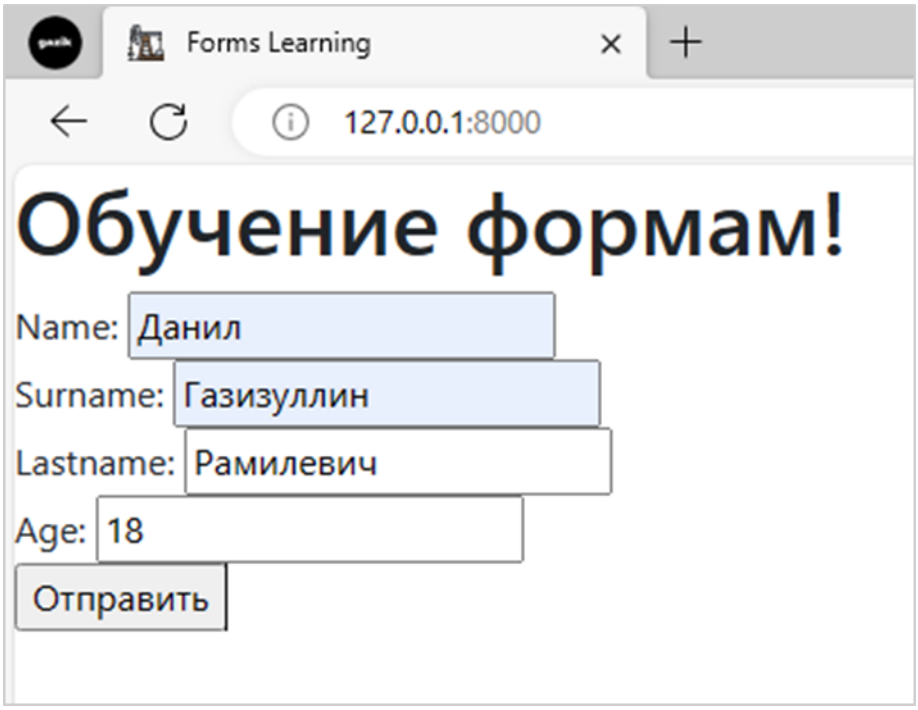


Рисунок 25- Отображение формы на странице (заполненной)

После отправки формы мы получим следующий результат (рисунок 26):

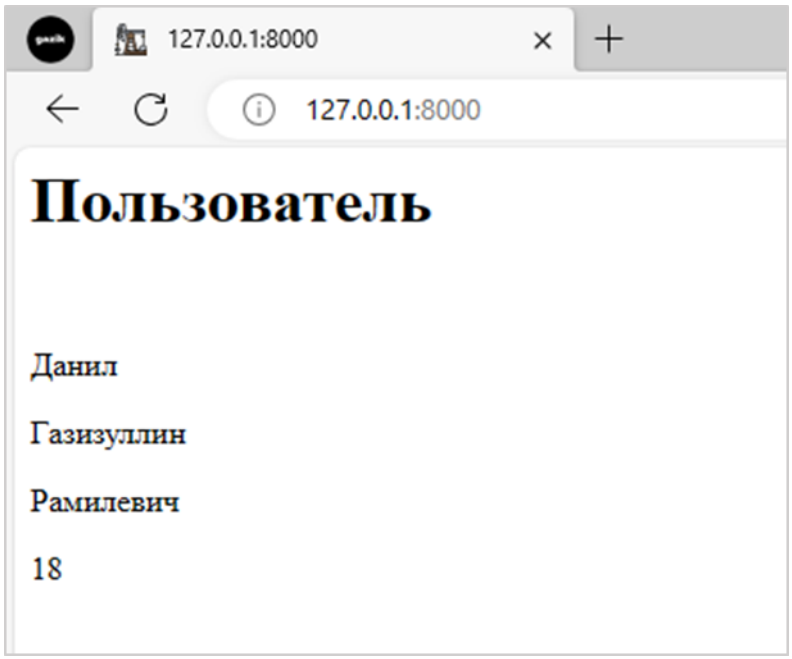


Рисунок 26 - HTTP-ответ от сервера

4.4 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте blog_site и приложении posts создайте форму добавления публикации с полями «Заголовок» и «Текст публикации». Реализуйте обработку введённых данных во views.py. После отправки формы сохраните публикацию и выведите введённые данные на отдельной странице результата.

Решение задачи:

1 models.py

```
from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    def __str__(self):
        return self.title
```

2 forms.py

```
from django import forms
from .models import Post
class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['title', 'content']
```

3 views.py

```
from django.shortcuts import render
from .forms import PostForm
def add_post(request):
```

```
    if request.method == "POST":
        form = PostForm(request.POST)
        if form.is_valid():
            post = form.save()
            return render(request, 'posts/result.html', {
                'post': post
            })
    else:
        form = PostForm()
    return render(request, 'posts/add_post.html', {
        'form': form
    })
```

4 urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('add/', views.add_post, name='add_post'),
]
```

5 add_post.html

```
{% extends 'base.html' %}
{% block content %}
<h2>Добавить публикацию</h2>
<form method="POST">
    {% csrf_token %}
    <div class="mb-3">
        <label>Заголовок</label>
        <input type="text" name="title" class="form-control">
    </div>
    <div class="mb-3">
        <label>Текст публикации</label>
        <textarea name="content" class="form-control"></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Сохранить</button>
</form>
{% endblock %}
```

6 result.html

```
{% extends 'base.html' %}
{% block content %}
<div class="card">
    <div class="card-body">
        <h3>Публикация добавлена</h3>
        <p><strong>Заголовок:</strong> {{ post.title }}</p>
        <p><strong>Текст:</strong> {{ post.content }}</p>
        <a href="{% url 'add_post' %}" class="btn btn-secondary">
            Добавить ещё
```

```
        </a>
    </div>
</div>
{% endblock %}
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте library_site и приложении books создайте форму добавления книги в шаблоне add_book.html. Реализуйте обработку POST-запроса во views.py и выведите введенное название книги на отдельной странице результата.

2 В ранее созданном проекте student_portal и приложении students создайте форму добавления студента с полями «Фамилия» и «Группа». Оформите форму с использованием классов Bootstrap form-control и btn btn-success.

3 В ранее созданном проекте movie_portal и приложении films создайте форму поиска фильма по названию с использованием GET-запроса. Выведите введенное значение на странице результата.

4 В ранее созданном проекте music_library и приложении albums создайте форму добавления музыкального альбома с полями «Название альбома» и «Исполнитель». Реализуйте обработку POST-запроса.

5 В ранее созданном проекте online_shop и приложении products создайте форму добавления товара с полями «Название товара» и «Цена». Используйте Bootstrap-классы для оформления формы.

6 В ранее созданном проекте news_portal и приложении news создайте форму добавления новости с полями «Заголовок» и «Текст новости». После отправки формы выведите введенные данные в шаблоне результата.

7 В ранее созданном проекте tourism_portal и приложении tours создайте форму поиска туристического маршрута по названию с использованием GET-запроса.

8 В ранее созданном проекте sport_club_site и приложении teams создайте форму регистрации спортивной команды с полями «Название команды» и «Тренер».

9 В ранее созданном проекте school_portal и приложении teachers создайте форму добавления преподавателя с полями «ФИО» и «Предмет». Добавьте кнопку отправки с классом btn btn-primary.

10 В ранее созданном проекте hospital_system и приложении patients создайте форму регистрации пациента с полями «ФИО» и «Возраст». Реализуйте обработку POST-запроса.

11 В ранее созданном проекте education_platform и приложении courses создайте форму записи на курс с полями «Имя слушателя» и «Название курса».

12 В ранее созданном проекте restaurant_site и приложении menu создайте форму добавления блюда с полями «Название блюда» и «Цена». Используйте Bootstrap для оформления элементов формы.

13 В ранее созданном проекте car_catalog и приложении cars создайте форму добавления автомобиля с полями «Марка автомобиля» и «Год выпуска». Реализуйте обработку введенных данных во views.py.

14 В ранее созданном проекте portfolio_site и приложении main создайте форму добавления проекта с полями «Название проекта», «Описание» и «Ссылка на проект». Используйте Bootstrap для оформления формы и элементов ввода. Реализуйте обработку введённых данных во views.py и вывод списка проектов на отдельной странице.

15 В ранее созданном проекте home_site и приложении home создайте форму добавления страницы сайта с полем «Название страницы» и «URL-адрес». Реализуйте обработку POST-запроса во views.py. Оформите вывод списка страниц с использованием Bootstrap-компонента list-group и добавьте навигационную панель (navbar) для перехода между страницами сайта.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите, что такое веб-форма в Django в языке программирования Python.

7.2 Объясните, для чего используется метод GET в языке программирования Python.

7.3 Объясните, для чего используется метод POST в языке программирования Python.

7.4 Охарактеризуйте, для чего используется объект request в Django в языке программирования Python.

7.5 Опишите, как передать данные из views.py в шаблон в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.